Krishi Mitra — Complete Deliverables Index

Batch 1: Numeric Safety + BM25 + Infrastructure

#	File	Destination	Closes	Description
1	`numeric_faithfulness.py`	`backend/eval/`	#5	Regex-based number+unit checker, cross-references answer vs context
2	`bm25_retriever.py`	`backend/eval/`	#10	Standalone BM25 (rank_bm25) on identical chunks, with RRF fusion
3	`run_retrieval_ablation.py`	`backend/eval/`	#10	Updated ablation: +BM25 configs, +bucketed exact-term vs semantic
4	`rag_service.py`	`backend/`	#5, #37, #38	Dead tavily removed, numeric checker wired, category filtering added
5	`threshold_sweep.py`	`backend/eval/`	#7	Empirically justifies HARD_REFUSAL_THRESHOLD using gold labels
6	`.env.example`	`backend/`	#6	Checked-in env template with all vars documented

Batch 2: LLM Replacement + Morphological Chunking

#	File	Destination	Closes	Description
7	`llm_client.py`	`backend/`	#1, #4	OpenRouter backend added, 4 hardcoded model strings removed
8	`vector_db.py`	`backend/`	#13	Field-aware chunking (one logical unit per chunk) + Indic NLP
9	`rag_service.py`	`backend/`	#1, #5, #37, #38	Uses embed_query wrapper, no hardcoded model fallback
10	`indic_preprocess.py`	`backend/`	#13	Kannada text normalization (Unicode NFC, ZWJ cleanup, spacing)
11	`.env.example`	`backend/`	#1, #6	OpenRouter vars, new default model `openai/gpt-oss-20b`

Documentation

File	Description
`CHANGES_SUMMARY.md`	Batch 1 integration guide + defense narrative
`CHANGES_SUMMARY_PART2.md`	Batch 2 integration guide + defense narrative

Generated by Kimi

Table 3 – continued

File	Description
`CHANGES_INDEX.md`	This file —complete index of all deliverables

Quick Start — Integration Order

Step 1: Copy all files

```
cd backend

# Batch 1 files
cp /path/to/numeric_faithfulness.py eval/
cp /path/to/bm25_retriever.py eval/
cp /path/to/run_retrieval_ablation.py eval/
cp /path/to/threshold_sweep.py eval/
cp /path/to/rag_service.py ./
cp /path/to/.env.example ./

# Batch 2 files
cp /path/to/llm_client.py ./
cp /path/to/vector_db.py ./
cp /path/to/indic_preprocess.py ./
# rag_service.py and .env.example from Batch 2 overwrite Batch 1 versions
```

Step 2: Install dependencies

```
pip install rank_bm25          # for BM25
pip install openai             # for OpenRouter backend
pip install indic-nlp-library  # optional, for full Kannada normalization
```

Step 3: Update .env

```
cp .env.example .env
# Edit .env:
#   - Set GROQ_API_KEY
#   - Set GENERATION_MODEL to active Groq model (e.g., openai/gpt-oss-20b)
#   - Optionally set OPENROUTER_API_KEY for cross-judge
```

Step 4: Test LLM client

```
python -c "from llm_client import call_llm; import asyncio;
↪  print(asyncio.run(call_llm([{'role':'user','content':'Hi'}], max_tokens=10)))"
```

Step 5: Rebuild database with field-aware chunking

```
# Stop main.py first
python vector_db.py
```

```
# Check chunk count and size distribution in output
```

Step 6: Relabel gold.jsonl (CRITICAL)

Field-aware chunking changes chunk boundaries and IDs. You must re-label `eval/gold.jsonl` manually. See CHANGES_SUMMARY_PART2.md for the dump command.

Step 7: Run evaluations

```
# Threshold justification
python eval/threshold_sweep.py

# BM25 ablation (with bucketed analysis)
python eval/run_retrieval_ablation.py

# Numeric faithfulness demo
python eval/numeric_faithfulness.py
```

Step 8: Start the app

```
python main.py
```

Coverage of Architect Review Gaps

Architect Gap	Status	How It' s Closed
No morphological preprocessing	✅	`indic_preprocess.py` + applied in `vector_db.py`
Fixed-size chunking ignores structure	✅	Field-aware chunking in `vector_db.py`
No conversational query rewriting	⏳	TODO #14 —not implemented yet
No metadata/category filtering	✅	Qdrant payload filter wired in `rag_service.py`
Single crop only	⏳	TODO #33 —future work
No streaming	⏳	TODO #15 —cut first if squeezed
Temperature 0.0 not documented	⏳	TODO #39 —report writing
No structured output for dosages	⏳	TODO #40 —Track 2
Only 1 of 3 models evaluated	🔄	`llm_client.py` ready for 3 backends, needs re-run
n=16 eval set too thin	⏳	TODO #18 —KCC dataset mining
No regression/CI harness	⏳	TODO #12 —GitHub Action
No numeric faithfulness check	✅	`numeric_faithfulness.py` wired in `rag_service.py`
No PII/prompt-injection handling	⏳	TODO #41 —report writing
No production readiness section	⏳	TODO #42 —report writing
Dead tavily_client code	✅	Removed from `rag_service.py`
Hardcoded model strings	✅	All removed, single `_DEFAULT_MODEL` in `llm_client.py`

Table 4 – continued

Architect Gap	Status	How It' s Closed
Groq deprecation	🔄	`llm_client.py` updated, needs API testing
Cross-judge capability	✅	OpenRouter backend added to `llm_client.py`

Legend: ✅ = implemented | 🔄 = code ready, needs testing/action | ⏳ = not yet done

---

What to Prioritize Next

If you have **2-3 hours**:

1. Test the new Groq model (update .env, run smoke test)
2. Rebuild database with field-aware chunking
3. Relabel `gold.jsonl`

If you have **1 day**: 4. Re-run full retrieval ablation (compare old vs new chunking) 5. Re-run generation eval with new model 6. Run threshold sweep and update `HARD_REFUSAL_THRESHOLD`

If you have **2 days**: 7. Add OpenRouter cross-judge to `run_eval.py` 8. Manual failure-mode review (TODO #11) 9. Write the "Design Choices" + "Production Readiness" report sections